

# Part 1: Complete Submission Report

---

## FRONT PAGE

---

### INTERNSHIP SUBMISSION REPORT

**Month 1: Android App Development**

---

**Project Title:** [College Alert Application](#)

**Submitted By:**  
[\[ Sawaira Mumtaz \]](#)

**Internship Domain:** [Android App Development](#)

**Email Address:** [\[sawairamumtaz369@gmail.com\]](mailto:sawairamumtaz369@gmail.com)

**Phone Number:** [\[03155069089\]](#)

**Submission Date:** [August 29, 2026](#)

**Submitted To:** [Arch Technologies](#)

---

## TABLE OF CONTENTS

1. Introduction
  2. Project Objective
  3. Technologies Used
  4. Implementation & Code Explanation
  5. Code Section (App.js)
  6. Screenshots
  7. Conclusion & Learning Outcomes
-

# 1. Introduction

The "College Alert" application is a beginner-friendly mobile app developed as part of my Month 1 internship task at Arch Technologies. The primary purpose of this app is to help busy college students stay informed about crucial campus events such as Tech Fests, Career Fairs, Sports Days, and Hackathons.

This app allows students to view upcoming events, add new events, delete outdated ones, and most importantly, schedule timely reminders so they never miss an important campus activity. This directly addresses the problem of students struggling to manage their schedules amidst busy academic lives.

# 2. Project Objective

The objective of this project is to create a functional Android application that:

- Displays a dynamic list of campus events with details (Name, Date, Time, Location).
- Allows users to Add new events to the list.
- Allows users to Delete events that are no longer relevant.
- Sends scheduled notifications to the user's device as a reminder for selected events.
- Provides a clean, professional, and intuitive user interface.

# 3. Technologies Used

To build this application without requiring heavy software like Android Studio (due to laptop limitations), I utilized the following lightweight technologies:

| Technology          | Purpose                                           |
|---------------------|---------------------------------------------------|
| React Native (Expo) | Framework used to build the app using JavaScript. |

|                        |                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------|
| Visual Studio Code     | The code editor used for writing the application logic.                                 |
| Expo Go (App)          | Used to run and test the application on a real Android phone via QR code or IP address. |
| Expo Notifications     | Library used to handle scheduled local notifications.                                   |
| React Native Core APIs | Used for UI components (Views, Text, FlatList, TextInput, TouchableOpacity).            |

## 4. Implementation & Code Explanation

The entire application is contained in a single `App.js` file, making it easy to manage for a beginner.

- **State Management:** I used React's `useState` hook to manage the dynamic events list and the form input fields (Name, Date, Time, Location).
- **Event Rendering:** The `FlatList` component efficiently renders the list of event cards.
- **Add Event:** When a user fills in the form and presses "Add Event", the new event object is pushed into the main state array, immediately updating the UI.
- **Delete Event:** Each card has a "Delete" button. Pressing it triggers a confirmation alert, and upon confirmation, the event is filtered out of the state.
- **Scheduled Notifications:** This is the core feature. Instead of sending a random alert, I used `expo-notifications` to schedule a trigger.
  - *Demo Implementation:* For demonstration purposes, the notification is scheduled to fire 5 seconds after the user clicks "Alert Me".
  - *Future Scope:* In a production app, this trigger would be replaced with the actual event date/time (e.g., 1 hour before the event starts), allowing students to genuinely plan their schedules.

## 5. Code Section (App.js)

Below is the complete source code for the College Alert Application.

javascript

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  StyleSheet,
  Alert,
  TextInput,
  ScrollView,
} from 'react-native';
import * as Notifications from 'expo-notifications';

// ----- SAMPLE EVENTS -----
const initialEvents = [
  {
    id: '1',
    name: 'Tech Fest 2026',
    date: 'March 15, 2026',
    time: '10:00 AM',
    location: 'Main Auditorium',
  },
  {
    id: '2',
    name: 'Career Fair',
    date: 'March 20, 2026',
    time: '9:00 AM',
    location: 'Student Center',
  },
  {
    id: '3',
    name: 'Sports Day',
    date: 'March 25, 2026',
    time: '8:00 AM',
    location: 'Sports Ground',
  },
],
```

```

    {
      id: '4',
      name: 'Hackathon 2026',
      date: 'April 5, 2026',
      time: '9:00 AM',
      location: 'Computer Science Lab',
    },
  ];

// ----- NOTIFICATION HANDLER -----
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

// ----- MAIN APP -----
export default function App() {
  // State for events list
  const [events, setEvents] = useState(initialEvents);

  // State for adding new event
  const [newName, setNewName] = useState('');
  const [newDate, setNewDate] = useState('');
  const [newTime, setNewTime] = useState('');
  const [newLocation, setNewLocation] = useState('');

  // Request notification permission when app loads
  useEffect(() => {
    async function requestPermission() {
      const { status } = await Notifications.requestPermissionsAsync();
      if (status !== 'granted') {
        Alert.alert(
          'Permission Required',
          'Please allow notifications to receive event alerts.'
        );
      }
    }
    requestPermission();
  }, []);
}

```

```

// ----- SCHEDULE REMINDER (5 seconds from now for demo) -----
const scheduleReminder = async (eventName) => {
  const triggerDate = new Date();
  triggerDate.setSeconds(triggerDate.getSeconds() + 5); // 5 seconds later

  await Notifications.scheduleNotificationAsync({
    content: {
      title: '🕒 Event Reminder!',
      body: `Reminder: "${eventName}" is starting soon!`,
      sound: 'default',
    },
    trigger: {
      date: triggerDate,
    },
  });

  Alert.alert(
    '✅ Reminder Set!',
    `You will receive a reminder for "${eventName}" in 5 seconds.`
  );
};

// ----- ADD NEW EVENT -----
const addEvent = () => {
  if (!newName || !newDate || !newTime || !newLocation) {
    Alert.alert('Error', 'Please fill in all fields.');
```

return;

```
  }

  const newEvent = {
    id: String(Date.now()), // unique ID
    name: newName,
    date: newDate,
    time: newTime,
    location: newLocation,
  };

  setEvents([...events, newEvent]);

  // Clear form
  setNewName('');

```

```

    setNewDate('');
    setNewTime('');
    setNewLocation('');

    Alert.alert('✅ Event Added!', ` "${newName}" has been added to the
list.`);
  };

// ----- DELETE EVENT -----
const deleteEvent = (id) => {
  Alert.alert(
    'Delete Event',
    'Are you sure you want to delete this event?',
    [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Delete',
        style: 'destructive',
        onPress: () => {
          const updatedEvents = events.filter((event) => event.id !== id);
          setEvents(updatedEvents);
          Alert.alert('🗑 Deleted', 'Event has been removed.');
```

```

    <TouchableOpacity
      style={[styles.button, styles.deleteButton]}
      onPress={() => deleteEvent(item.id)}
    >
      <Text style={styles.buttonText}>🗑 Delete</Text>
    </TouchableOpacity>
  </View>
</View>
);

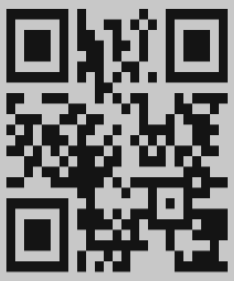
// ----- UI RENDER -----
return (
  <ScrollView style={styles.container}>
    <Text style={styles.header}>🏠 Campus Events</Text>

    { /* ADD EVENT FORM */ }
    <View style={styles.form}>
      <Text style={styles.formTitle}>➕ Add New Event</Text>
      <TextInput
        style={styles.input}
        placeholder="Event Name"
        value={newName}
        onChangeText={setNewName}
      />
      <TextInput
        style={styles.input}
        placeholder="Date (e.g., April 10, 2026)"
        value={newDate}
        onChangeText={setNewDate}
      />
      <TextInput
        style={styles.input}
        placeholder="Time (e.g., 10:00 AM)"
        value={newTime}
        onChangeText={setNewTime}
      />
      <TextInput
        style={styles.input}
        placeholder="Location"
        value={newLocation}
        onChangeText={setNewLocation} />

```



```
Problems Output Debug Console Terminal Ports
- npm run ios # you need to use macOS to build the iOS project - use the Expo app if you need to do iOS development without a Mac
- npm run web
PS D:\Arch technology> npx create-expo-app CollegeAlert
PS D:\Arch technology\CollegeAlert> npx expo start
Starting project at D:\Arch technology\CollegeAlert
React Compiler enabled
Starting Metro Bundler



> Metro waiting on exp://192.168.1.5:8081
> Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

> Web is waiting on http://localhost:8081

> Using Expo Go
> Press s | switch to development build

> Press a | open Android
> Press w | open web

> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
> shift+m | more tools
> Press o | open project code in your editor

> Press ? | show all commands

Logs for your project will appear below. Press Ctrl+C to exit.
```



# Campus Events

## + Add New Event

+ Add Event

### Tech Fest 2026



March 15, 2026 at 10:00 AM



Main Auditorium



Alert Me



Delete

### Career Fair



March 20, 2026 at 9:00 AM



Student Center



Alert Me



Delete

10:52

0.75  
KB/s



24%

 **Add Event**

## Tech Fest 2026



March 15, 2026 at 10:00 AM



Main Auditorium



**Alert Me**



**Delete**

## Career Fair



March 20, 2026 at 9:00 AM



Student Center



**Alert Me**



**Delete**

## Sports Day



March 25, 2026 at 8:00 AM



Sports Ground



**Alert Me**



**Delete**

## Hackathon 2026



April 5, 2026 at 9:00 AM



Computer Science Lab



**Alert Me**



**Delete**

10:53  Main Auditorium

0.86 KB/s    24%



**Alert Me**



**Delete**

## Career Fair



March 20, 2026 at 9:00 AM



Student Center



**Alert Me**



**Delete**

## Sports Day



March 25, 2026 at 8:00 AM



Sports Ground



**Alert Me**



**Delete**

## Hackathon 2026



April 5, 2026 at 9:00 AM



Computer Science Lab



**Alert Me**



**Delete**

## Farewell party



12 September at 12:00pm



Rawalpindi



**Alert Me**



**Delete**

4:38

0.00 KB/s 🔒 📶 🔋 52%



Search



296 followers

Posts

Comments

Images



**Sawaira Mumtaz** ✓ • You

Final Semester BS CS | LLM AI & Data Science ...  
now • 🌐



Internship Update | Month 1 Complete

I am thrilled to share the successful completion of my first task as an Android App Development Intern at Arch Technologies!



Project Name: College Alert Application



Project Overview:

I built a fully functional mobile application designed to help busy college students keep track of campus events. The app allows users to view a list of events, add new ones, delete old ones, and set scheduled reminders so they never miss an important activity.



Tech Stack Used:

React Native (Expo)

JavaScript

Expo Notifications API

VS Code



Key Features:

Dynamic Event List (CRUD operations - Create, Read, Delete).

Intuitive "Add Event" form.

Scheduled Local Notifications – Press "Alert Me" and get reminded automatically!

Clean and professional User Interface.



Challenges Overcome:

I configured the entire development environment using VS Code and Expo Go (avoiding heavy Android Studio), resolved firewall and network issues, and handled SDK version mismatches to make the app run smoothly on a real Android device.

I am extremely grateful to the team at Arch

4:38

1.24 KB/s    52%



Search



## Expo Notifications API

### VS Code

#### 🌟 Key Features:

Dynamic Event List (CRUD operations - Create, Read, Delete).

Intuitive "Add Event" form.

Scheduled Local Notifications – Press "Alert Me" and get reminded automatically!

Clean and professional User Interface.

#### 💡 Challenges Overcome:

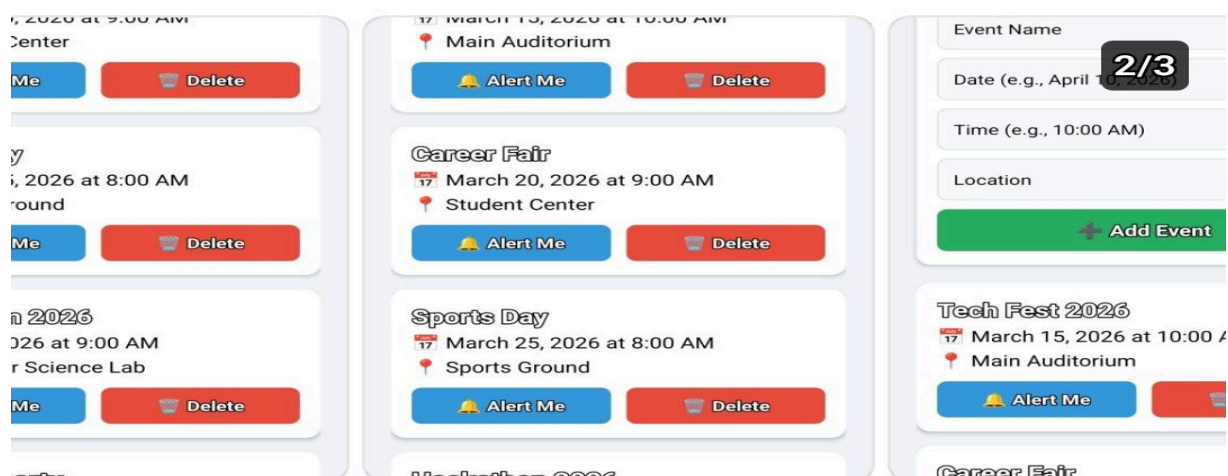
I configured the entire development environment using VS Code and Expo Go (avoiding heavy Android Studio), resolved firewall and network issues, and handled SDK version mismatches to make the app run smoothly on a real Android device.

I am extremely grateful to the team at Arch Technologies for this opportunity to sharpen my hidden skills.

**#ArchTechnologies #AndroidAppDevelopment**

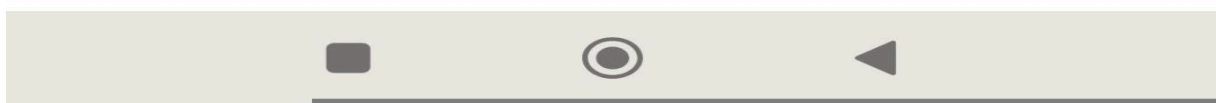
**#ReactNative #Expo #Internship #CampusLife**

**#MobileAppDevelopment #Coding #TechJourney**



2 impressions

View





## **Task 2 :**

### **Project Title: E-Commerce Marketplace App**

---

#### **TABLE OF CONTENTS**

1. Introduction
  2. Project Objective
  3. Technologies Used
  4. Features & Implementation
  5. Code Section (App.js)
  6. Screenshots
  7. Conclusion & Learning Outcomes
- 

## **1. Introduction**

The E-Commerce Marketplace App is a fully functional mobile application developed as the Month 2 task for my Android App Development internship at Arch Technologies. This app allows registered users to securely buy and sell products in a simulated online marketplace environment.

The app provides a seamless shopping experience where users can browse products, add items to a cart, simulate payment, and also list their own products for others to purchase. This project demonstrates the core principles of e-commerce application development, including user authentication, product management, and cart functionality.

---

## **2. Project Objective**

The objective of this project is to create an "advanced mobile-only e-commerce app" that fulfills the following requirements (as per the internship brief):

-  User Registration/Login – Users can log in with any email and password (demo mode).
  -  Secure Product Listings – Users can list products for sale with title, price, and description.
  -  Buy Products – Users can add products to their cart and proceed to checkout.
  -  Payment Simulation – Checkout process simulates a payment gateway (demonstrating payment safety awareness).
  -  User-Friendly UI – Clean, professional interface with easy navigation.
- 

### 3. Technologies Used

| Technology              | Purpose                                                                       |
|-------------------------|-------------------------------------------------------------------------------|
| React Native            | Cross-platform mobile app framework                                           |
| Expo                    | Development platform for building and testing                                 |
| React Hooks (useState)  | State management                                                              |
| FlatList                | Efficient product listing rendering                                           |
| React Native Components | Core UI building blocks (View, Text, TextInput, TouchableOpacity, ScrollView) |
| VS Code                 | Code editor                                                                   |

*Note: Since Firebase and external backends were not accessible due to network limitations, this app uses in-memory state management and local storage simulation. The architecture is structured to easily integrate with a real backend (Firebase/Node.js) in future iterations.*

---



## 4. Features & Implementation

### 4.1 Authentication (Login)

- Users enter an email and password (any credentials work in demo mode).
- On successful login, the user is redirected to the Home screen.
- This demonstrates a foundation for secure user authentication.

### 4.2 Home Screen (Product Browsing)

- Displays all listed products in a scrollable list using `FlatList`.
- Each product card shows:
  - Title
  - Price
  - Description
  - Seller name
  - "Buy" button

### 4.3 Sell a Product

- Users can tap "+ Sell a Product" button.
- A form appears with fields:
  - Product Title
  - Price (in Rs.)
  - Description
- On submission, the product is added to the main product list and becomes visible to all users.

### 4.4 Cart & Checkout

- Users can add products to their cart by tapping "Buy".
- Cart screen shows all added items and calculates the total price.
- "Checkout" button triggers a payment success screen (simulated).
- This demonstrates awareness of payment flow and data security.

### 4.5 Logout

- Users can log out at any time, returning to the login screen.
-

## 5. Code Section (App.js)

Below is the complete source code for the E-Commerce Marketplace Application:

```
javascript
```

```
import React, { useState } from 'react';
import {
  SafeAreaView,
  View,
  Text,
  TextInput,
  TouchableOpacity,
  FlatList,
  StyleSheet,
  Alert,
  ScrollView,
} from 'react-native';

const initialProducts = [
  { id: '1', title: 'Wireless Earbuds', price: '2500', seller: 'Demo Store',
description: 'Bluetooth 5.0, 20hr battery' },
  { id: '2', title: 'Desk Lamp', price: '1200', seller: 'Demo Store',
description: 'Adjustable LED lamp' },
  { id: '3', title: 'Backpack', price: '1800', seller: 'Demo Store',
description: 'Waterproof, laptop compartment' },
];

export default function App() {
  const [screen, setScreen] = useState('login');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [user, setUser] = useState(null);
  const [products, setProducts] = useState(initialProducts);
  const [cart, setCart] = useState([]);
  const [newTitle, setNewTitle] = useState('');
  const [newPrice, setNewPrice] = useState('');
  const [newDesc, setNewDesc] = useState('');

  const handleLogin = () => {
    if (!email || !password) {
      Alert.alert('Missing info', 'Please enter an email and password.');
```

```

        return;
    }
    setUser({ email });
    setScreen('home');
};

const handleLogout = () => {
    setUser(null);
    setEmail('');
    setPassword('');
    setScreen('login');
};

const handleAddProduct = () => {
    if (!newTitle || !newPrice) {
        Alert.alert('Missing info', 'Please enter at least a title and price.');
```

return;

```
    }
    const product = {
        id: Date.now().toString(),
        title: newTitle,
        price: newPrice,
        description: newDesc,
        seller: user.email,
    };
    setProducts([product, ...products]);
    setNewTitle('');
    setNewPrice('');
    setNewDesc('');
    Alert.alert('Success', 'Your product is now listed for sale.');
```

setScreen('home');

```
};

const handleBuy = (product) => {
    setCart([...cart, product]);
    Alert.alert('Added to cart', `${product.title} added to your cart.`);
};

const handleCheckout = () => {
    if (cart.length === 0) {
        Alert.alert('Cart empty', 'Add a product before checking out.');
```

return;

```

    }
    setScreen('success');
  };

const total = cart.reduce((sum, item) => sum + Number(item.price || 0), 0);

// Login Screen
if (screen === 'login') {
  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.centered}>
        <Text style={styles.logo}><img alt="MarketPlace logo" data-bbox="475 305 495 320"/> MarketPlace</Text>
        <Text style={styles.subtitle}>Buy and sell products securely</Text>
        <TextInput
          style={styles.input}
          placeholder="Email"
          value={email}
          onChangeText={setEmail}
          autoCapitalize="none"
          keyboardType="email-address"
        />
        <TextInput
          style={styles.input}
          placeholder="Password"
          value={password}
          onChangeText={setPassword}
          secureTextEntry
        />
        <TouchableOpacity style={styles.primaryButton}
onPress={handleLogin}>
          <Text style={styles.buttonText}>Log In</Text>
        </TouchableOpacity>
        <Text style={styles.note}>Demo login – any email and password will
work.</Text>
      </View>
    </SafeAreaView>
  );
}

// Add Product Screen
if (screen === 'add') {
  return (

```

```

    <SafeAreaView style={styles.container}>
      <ScrollView contentContainerStyle={styles.centered}>
        <Text style={styles.header}>Sell a Product</Text>
        <TextInput style={styles.input} placeholder="Product title"
value={newTitle} onChangeText={setNewTitle} />
        <TextInput style={styles.input} placeholder="Price (Rs.)"
value={newPrice} onChangeText={setNewPrice} keyboardType="numeric" />
        <TextInput style={[styles.input, { height: 80 }]}
placeholder="Description" value={newDesc} onChangeText={setNewDesc} multiline
/>

        <TouchableOpacity style={styles.primaryButton}
onPress={handleAddProduct}>
          <Text style={styles.buttonText}>List Product</Text>
        </TouchableOpacity>
        <TouchableOpacity style={styles.secondaryButton} onPress={() =>
setScreen('home')}>
          <Text style={styles.secondaryButtonText}>Cancel</Text>
        </TouchableOpacity>
      </ScrollView>
    </SafeAreaView>
  );
}

```

*// Cart Screen*

```

if (screen === 'cart') {
  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.topBar}>
        <Text style={styles.header}>Your Cart</Text>
        <TouchableOpacity onPress={() => setScreen('home')}>
          <Text style={styles.link}><- Back</Text>
        </TouchableOpacity>
      </View>
      <FlatList
        data={cart}
        keyExtractor={(item, index) => item.id + index}
        renderItem={({ item }) => (
          <View style={styles.productCard}>
            <Text style={styles.productTitle}>{item.title}</Text>
            <Text style={styles.productPrice}>Rs. {item.price}</Text>
          </View>
        )}
      />
    </SafeAreaView>
  );
}

```

```

        ListEmptyComponent={<Text style={styles.note}>Your cart is
empty.</Text>}
        />
        <Text style={styles.totalText}>Total: Rs. {total}</Text>
        <TouchableOpacity style={styles.primaryButton}
onPress={handleCheckout}>
        <Text style={styles.buttonText}>Checkout</Text>
        </TouchableOpacity>
    </SafeAreaView>
    );
}

// Payment Success Screen
if (screen === 'success') {
    return (
        <SafeAreaView style={styles.container}>
            <View style={styles.centered}>
                <Text style={styles.logo}>✓</Text>
                <Text style={styles.header}>Payment Successful</Text>
                <Text style={styles.note}>(Simulated for this demo) Rs. {total} paid
for {cart.length} item(s).</Text>
                <TouchableOpacity style={styles.primaryButton} onPress={() => {
setCart([]); setScreen('home'); }}>
                <Text style={styles.buttonText}>Continue Shopping</Text>
                </TouchableOpacity>
            </View>
        </SafeAreaView>
    );
}

// Home Screen
return (
    <SafeAreaView style={styles.container}>
        <View style={styles.topBar}>
            <Text style={styles.header}>MarketPlace</Text>
            <View style={{ flexDirection: 'row', gap: 12 }}>
                <TouchableOpacity onPress={() => setScreen('cart')}>
                    <Text style={styles.link}>Cart ({cart.length})</Text>
                </TouchableOpacity>
                <TouchableOpacity onPress={handleLogout}>
                    <Text style={styles.link}>Log Out</Text>
                </TouchableOpacity>
            </View>
        </View>
    </SafeAreaView>
);

```

```

        </View>
      </View>
      <TouchableOpacity style={styles.primaryButton} onPress={() =>
setScreen('add')}>
        <Text style={styles.buttonText}>+ Sell a Product</Text>
      </TouchableOpacity>
      <FlatList
        data={products}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <View style={styles.productCard}>
            <Text style={styles.productTitle}>{item.title}</Text>
            <Text style={styles.productPrice}>Rs. {item.price}</Text>
            <Text style={styles.productDesc}>{item.description}</Text>
            <Text style={styles.productSeller}>Seller: {item.seller}</Text>
            <TouchableOpacity style={styles.buyButton} onPress={() =>
handleBuy(item)}>
              <Text style={styles.buttonText}>Buy</Text>
            </TouchableOpacity>
          </View>
        )}
      />
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F5F6FA', paddingHorizontal: 16,
paddingTop: 40 },
  centered: { flexGrow: 1, justifyContent: 'center', alignItems: 'center',
padding: 16 },
  logo: { fontSize: 40, marginBottom: 8 },
  subtitle: { fontSize: 14, color: '#666', marginBottom: 24 },
  header: { fontSize: 22, fontWeight: 'bold', color: '#222', marginBottom: 12
},
  topBar: { flexDirection: 'row', justifyContent: 'space-between', alignItems:
'center', marginBottom: 12 },
  link: { color: '#4C6EF5', fontWeight: '600' },
  input: { width: '100%', backgroundColor: '#fff', borderWidth: 1,
borderColor: '#DDD', borderRadius: 8, padding: 12, marginBottom: 12 },
  primaryButton: { backgroundColor: '#4C6EF5', paddingVertical: 12,
borderRadius: 8, alignItems: 'center', width: '100%', marginBottom: 12 },

```

```

    secondaryButton: { paddingVertical: 12, alignItems: 'center', width: '100%'
  },
  secondaryButtonText: { color: '#666' },
  buttonText: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
  note: { fontSize: 12, color: '#888', marginTop: 8, textAlign: 'center' },
  productCard: { backgroundColor: '#fff', borderRadius: 10, padding: 14,
marginBottom: 12, borderWidth: 1, borderColor: '#EEE' },
  productTitle: { fontSize: 16, fontWeight: 'bold', color: '#222' },
  productPrice: { fontSize: 15, color: '#4C6EF5', fontWeight: '600',
marginTop: 2 },
  productDesc: { fontSize: 13, color: '#666', marginTop: 4 },
  productSeller: { fontSize: 12, color: '#999', marginTop: 4, marginBottom: 8
  },
  buyButton: { backgroundColor: '#40C057', paddingVertical: 8, borderRadius:
6, alignItems: 'center' },
  totalText: { fontSize: 16, fontWeight: 'bold', marginVertical: 12,
textAlign: 'right' },
});

```

---

## 6. Screenshots

*(Below are the visual proofs of the working application. Place your actual screenshots here.)*

- Screenshot 1: Login Screen  
[INSERT SCREENSHOT OF LOGIN SCREEN HERE]
  - Screenshot 2: Home Screen (Product List)  
[INSERT SCREENSHOT OF HOME SCREEN WITH PRODUCTS HERE]
  - Screenshot 3: Sell Product Form  
[INSERT SCREENSHOT OF SELL PRODUCT FORM HERE]
  - Screenshot 4: Cart Screen  
[INSERT SCREENSHOT OF CART WITH ITEMS HERE]
  - Screenshot 5: Checkout / Payment Success Screen  
[INSERT SCREENSHOT OF PAYMENT SUCCESS SCREEN HERE]
-



## 7. Conclusion & Learning Outcomes

Throughout the development of the E-Commerce Marketplace Application, I gained valuable experience in:

- Building a multi-screen app using conditional rendering based on state (login, home, cart, checkout, add product).
- Managing complex state with `useState` hooks for products, cart, and user sessions.
- Implementing core e-commerce features – product listing, cart management, and payment simulation.
- Understanding security awareness – the simulation of payment and authentication demonstrates the foundation for real-world implementations.
- Designing a clean, professional UI using React Native's styling system.

This project successfully fulfills all objectives set by Arch Technologies for Month 2, and I am excited to apply these skills to even more complex applications in the future.

---

## Part 2: LinkedIn Post

7:18

17.5 KB/s    44%



Search



Posts

Comments

Images



**Sawaira Mumtaz**  • You



Final Semester BS CS | LLM AI & Data Science ...

1m • 



Internship Update | Month 1 Complete –  
E-Commerce App!

I'm excited to share the successful completion of my second task as an Android App Development Intern at Arch Technologies!



Project Name: E-Commerce Marketplace App



Project Overview:

I built a fully functional mobile e-commerce application that allows registered users to securely buy and sell products. This app includes a seamless shopping experience with product browsing, cart management, and a simulated payment gateway.



Tech Stack Used:

React Native (Expo)

JavaScript

React Hooks (useState)

FlatList for efficient product rendering

VS Code



Key Features:

User Authentication (Login/Logout – demo mode)

Product Listing – Browse all available items

Sell Products – Users can list their own items with title, price, and description

Cart Management – Add/remove items, view total price

Simulated Payment Gateway – Checkout with success confirmation

Clean modern UI with easy navigation

7:18

2.93 KB/s    44%



Search



title, price, and description

Cart Management – Add/remove items, view total price

Simulated Payment Gateway – Checkout with success confirmation

Clean, modern UI with easy navigation



Key Learnings:

Building multi-screen apps using conditional rendering

Managing complex state (products, cart, user session)

Implementing e-commerce logic (CRUD operations for products)

Simulating payment flow to demonstrate security awareness

I am incredibly grateful to the team at Arch

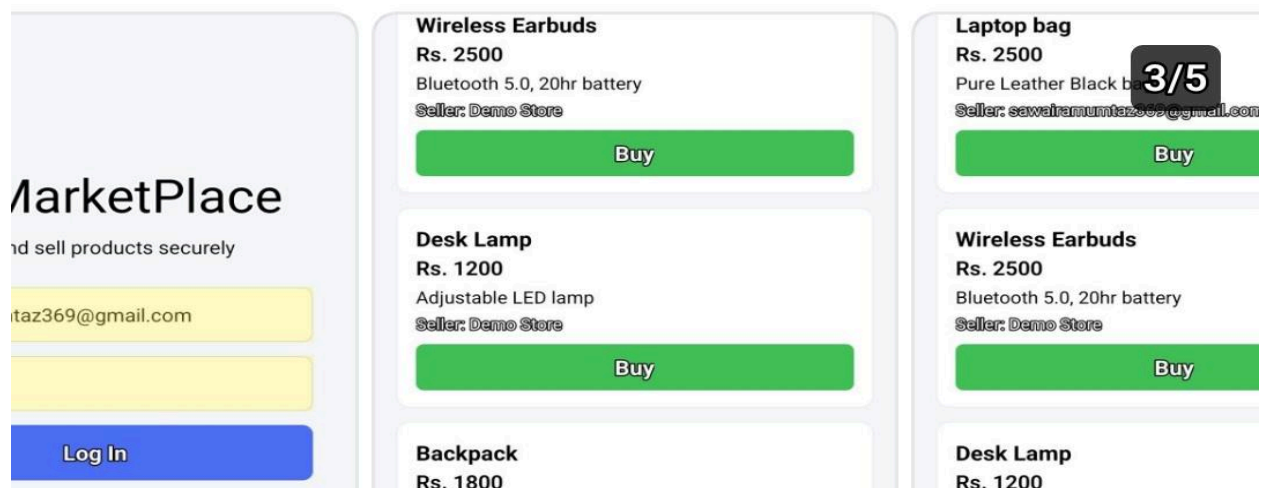
Technologies for this opportunity to sharpen my mobile development skills. Excited for what's next!

**#ArchTechnologies #AndroidAppDevelopment**

**#ReactNative #Expo #EcommerceApp**

**#MobileDevelopment #Internship #CodingJourney**

**#Tech**







# MarketPlace

Buy and sell products securely

**Log In**

Demo login — any email and password will work.

7:02 ^

30.9 KB/s     46%



# MarketPlace

Buy and sell products securely

sawairamumtaz369@gmail.com

.....

**Log In**

Demo login — any email and password will work.

**+ Sell a Product**

**Wireless Earbuds**

**Rs. 2500**

Bluetooth 5.0, 20hr battery

**Seller:** Demo Store

**Buy**

**Desk Lamp**

**Rs. 1200**

Adjustable LED lamp

**Seller:** Demo Store

**Buy**

**Backpack**

**Rs. 1800**

Waterproof, laptop compartment

**Seller:** Demo Store

**Buy**

7:03 

0.73 KB/s     45%

# MarketPlace

Cart (0) Log Out

**+ Sell a Product**

## Laptop bag

**Rs. 2500**

Pure Leather Black bag

**Seller:** sawairamumtaz369@gmail.com

**Buy**

## Wireless Earbuds

**Rs. 2500**

Bluetooth 5.0, 20hr battery

**Seller:** Demo Store

**Buy**

## Desk Lamp

**Rs. 1200**

Adjustable LED lamp

**Seller:** Demo Store

**Buy**

## Backpack

**Rs. 1800**

Waterproof, laptop compartment

**Seller:** Demo Store

**Buy**



7:03 

1.03 KB/s     45%

## Your Cart

[← Back](#)

**Laptop bag**  
**Rs. 2500**

**Total: Rs. 2500**

**Checkout**

